

CS57300
PURDUE UNIVERSITY

DATA MINING

DATA MINING COMPONENTS

- ▶ Task specification: **Prediction**
- ▶ Knowledge representation
- ▶ Learning technique
- ▶ Prediction and/or interpretation

PREDICTIVE MODELING: LEARNING

LEARNING PREDICTIVE MODELS

- ▶ Select a **knowledge representation** (a “model”)
 - ▶ Defines a **space** of possible models $M=\{M_1, M_2, \dots, M_k\}$
- ▶ Define **scoring functions** to “score” different models
- ▶ Use **search** to identify “best” model(s)
 - ▶ Search the space of models (i.e., with alternative structures and/or parameters)
 - ▶ Evaluate possible models with **scoring function** to determine the model which best fits the data
 - ▶ Score function can be used to search over **parameters** and/or **model structure**

PREDICTIVE SCORING FUNCTIONS

- ▶ Assess the quality of predictions for a set of instances
- ▶ Measures **difference** between the prediction M makes for an instance i and the true class label value of i

$$S(M) = \sum_{i=1}^N d[f(x(i); M), y(i)]$$

The diagram illustrates the components of the predictive scoring function $S(M)$. It features the equation $S(M) = \sum_{i=1}^N d[f(x(i); M), y(i)]$ with four colored arrows pointing to specific parts: an orange arrow points to the summation symbol $\sum$ with the label "Sum over examples"; a green arrow points to the distance function d with the label "Distance between predicted and true"; a blue arrow points to the predicted class label $f(x(i); M)$ with the label "Predicted class label for item i "; and a red arrow points to the true class label $y(i)$ with the label "True class label for item i ".

PREDICTIVE SCORING FUNCTIONS

- ▶ Common score functions:

- ▶ Zero-one loss

$$S_{0/1}(M) = \frac{1}{N} \sum_{i=1}^N I[f(x(i); M), y(i)]$$

$$\text{where } I(a, b) = \begin{cases} 1 & a \neq b \\ 0 & \text{otherwise} \end{cases}$$

- ▶ Squared loss

$$S_{sq}(M) = \frac{1}{N} \sum_{i=1}^N [f(x(i); M) - y(i)]^2$$

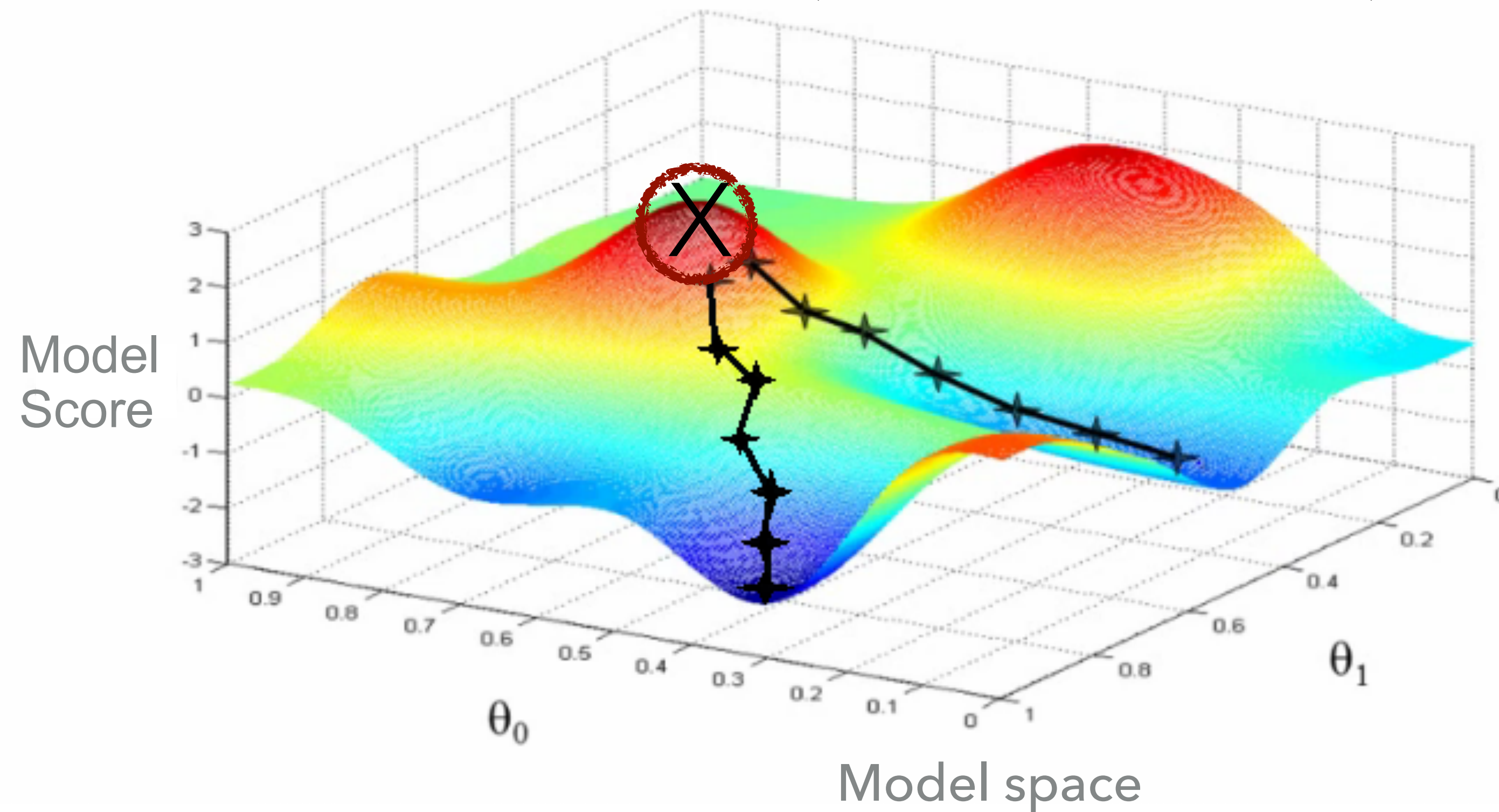
- ▶ Do we minimize or maximize these functions?

SEARCHING OVER MODELS

- ▶ Consider a **space** of possible models $M=\{M_1, M_2, \dots, M_k\}$ with parameters θ
- ▶ Search could be over model structures or parameters, e.g.:
 - ▶ **Parameters:** In a perceptron, find the weights (**w**) that minimize 0-1 loss
 - ▶ **Model structure:** In decision trees, find the tree structure that maximizes accuracy on the training data

WHAT SPACE ARE WE SEARCHING?

Learned model $\approx (\theta_0 = 0.8, \theta_1 = 0.4)$



OPTIMIZATION OVER SCORE FUNCTIONS

▶ **Smooth** functions:

- ▶ If a function is *smooth*, it is differentiable and the derivatives are continuous, then we can use gradient-based optimization
 - ▶ If function is *convex*, we can solve the minimization problem in closed form: $\nabla S(\theta)$ using **convex optimization**
 - ▶ If function is smooth but not convex/concave, we can use iterative search over the surface of S to find a local minimum (e.g., hill-climbing)

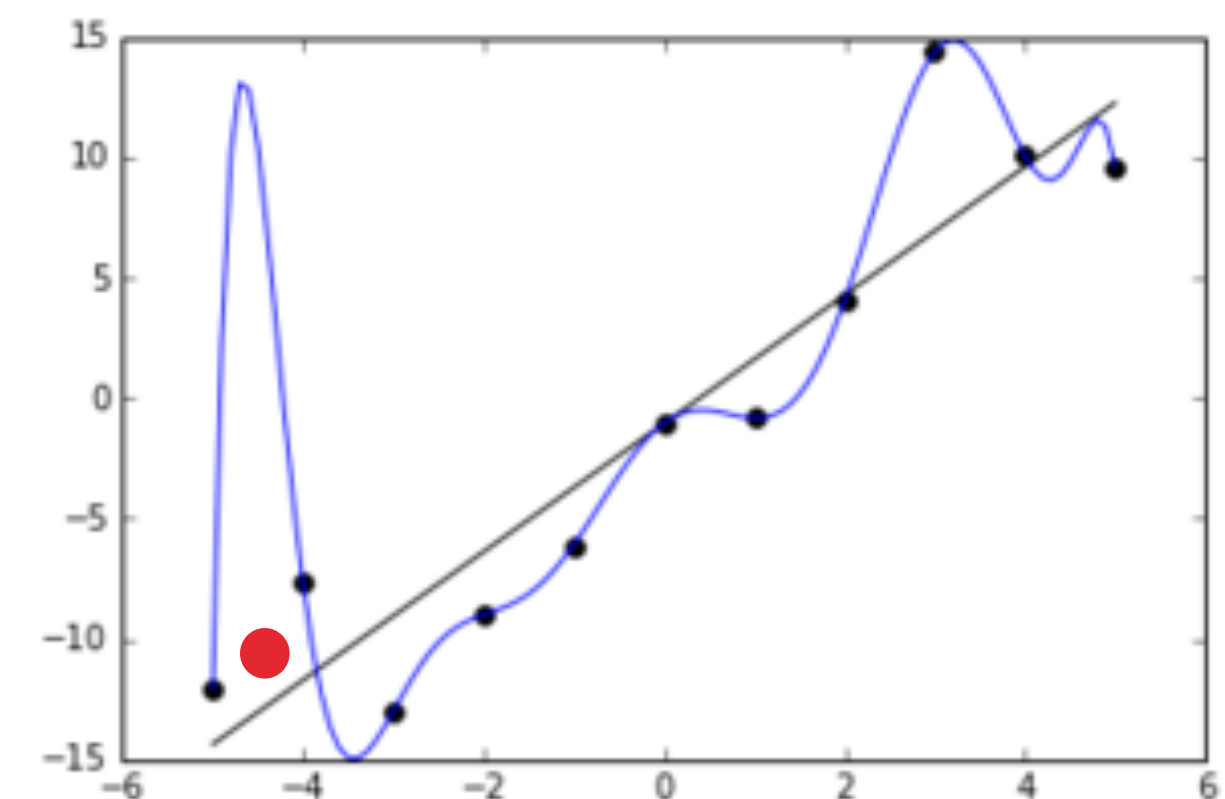
▶ **Non-smooth** functions:

- ▶ If the function is *discrete*, then traditional optimization methods that rely on smoothness are not applicable. Instead we need to use **combinatorial optimization**

PREDICTION AND EVALUATION

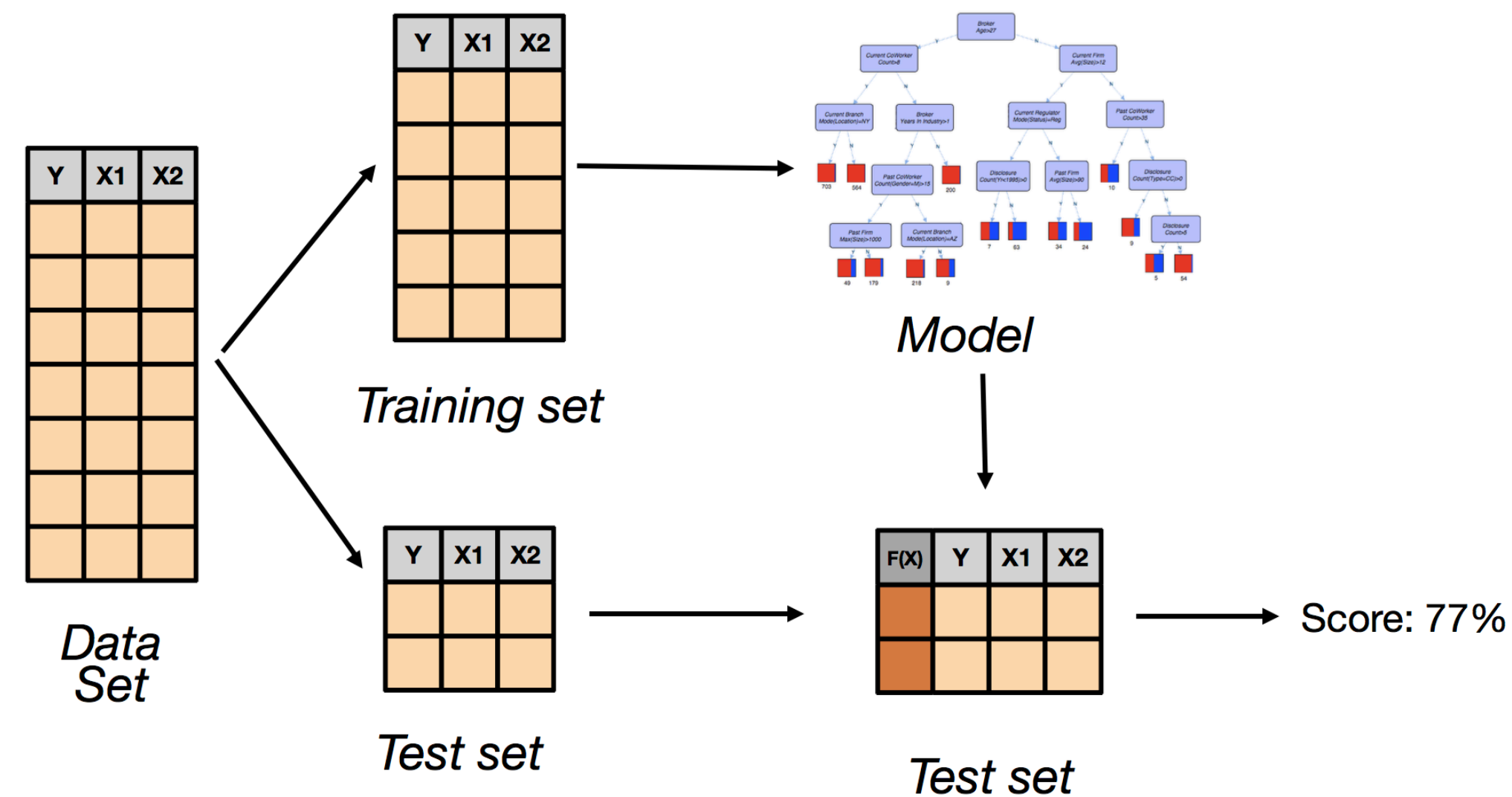
PREDICTING UNSEEN DATA

- ▶ Use the learned model to predict class label y for unseen data x
- ▶ Evaluating the performance of the learned model (i.e., its capability to generalize) by computing the scoring functions on the unseen data
- ▶ How to evaluate the performance of a model given a limited amount of data?
 - ▶ Can we train the model on the entire dataset and use the scoring function value on it as an estimate for the model's performance on the unseen data?



SPLIT INTO TRAINING DATA AND TESTING DATA

- ▶ Split the dataset into disjoint two sets: training set and testing set
- ▶ Learn the model using the training set and evaluate the model's performance on testing set



OVERFITTING

- ▶ When the performance of your model on the training data is much better than its performance on the testing data, you are likely overfitting...
- ▶ Consider a distribution D of data representing a population and a sample D_S drawn from D , which is used as training data
- ▶ Given a model space M , a score function S , and a learning algorithm that returns a model $m \in M$, the algorithm **overfits** the training data D_S if:
 $\exists m' \in M$ such that $S(m, D_S) > S(m', D_S)$ but $S(m, D) < S(m', D)$
 - ▶ In other words, there is another model (m') that is better on the entire distribution and if we had learned from the full data we would have selected it instead

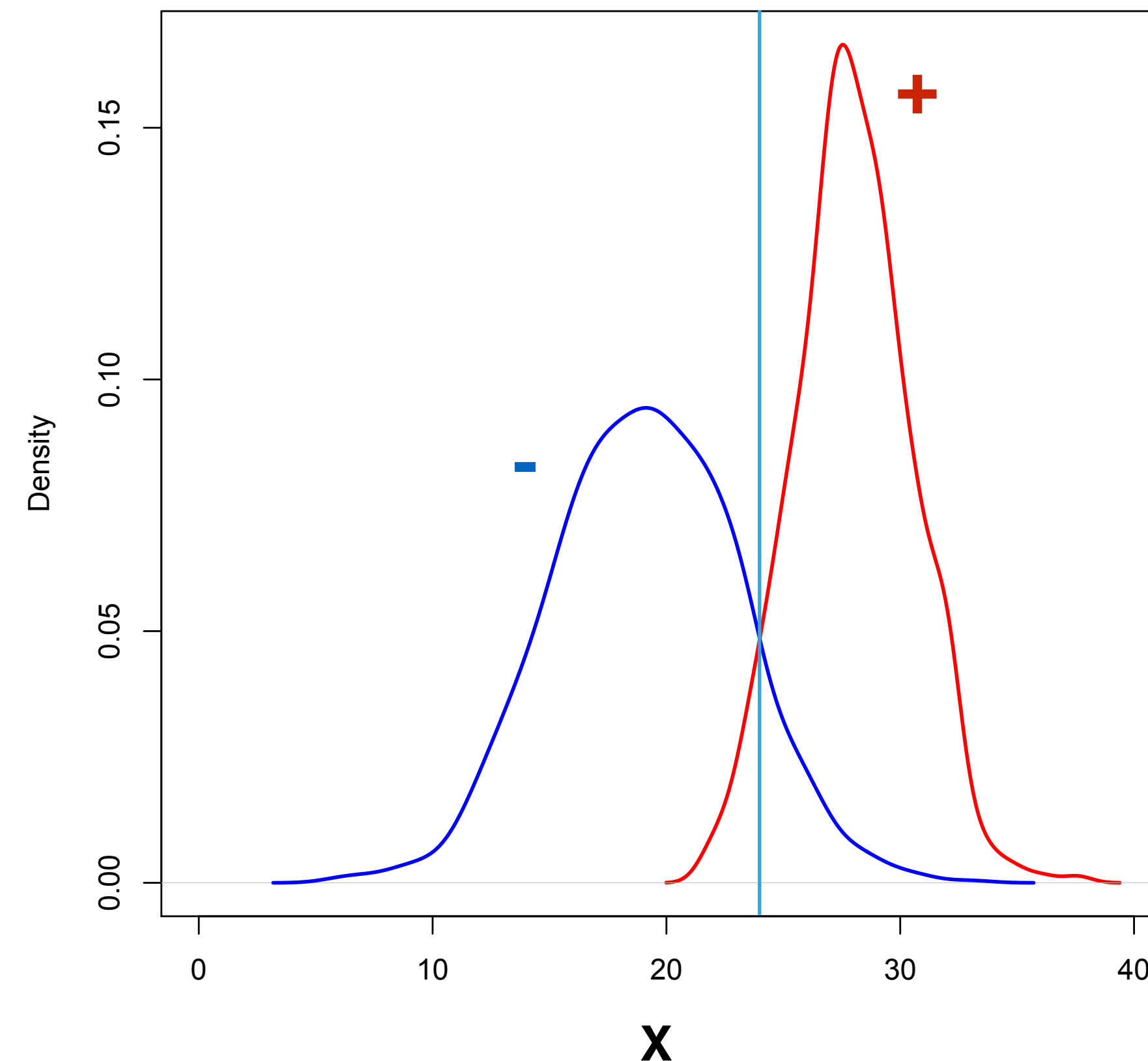
EXAMPLE LEARNING PROBLEM

Knowledge representation:
If-then rules

Example rule:
If $x > 24$ then +
Else -

What is the model space?

All possible thresholds



Task: Devise a rule to classify items based on the attribute **X**

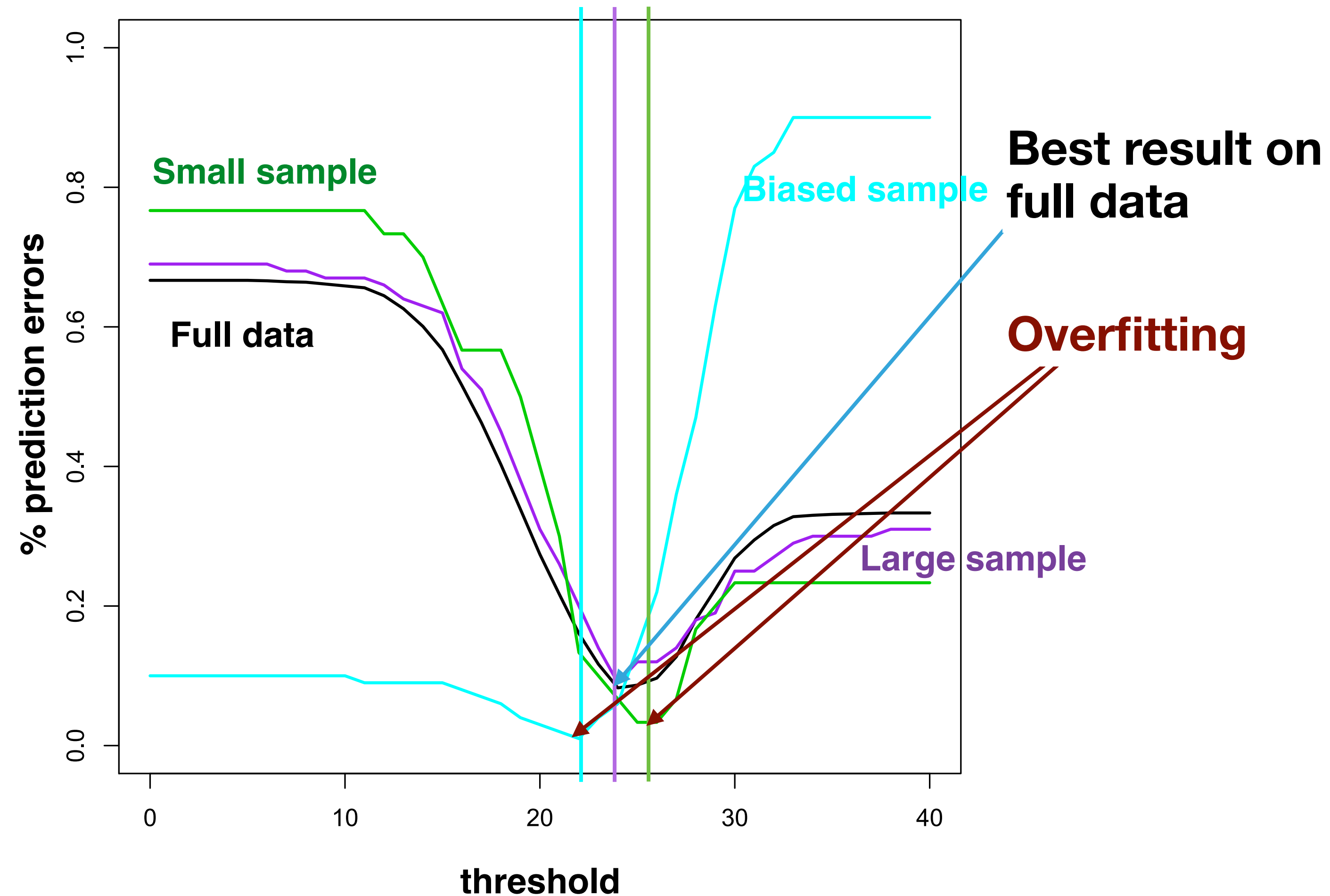
What score function?

Prediction error rate

SCORE FUNCTION OVER MODEL SPACE

Search procedure?

Try all thresholds, select one with lowest score



DEAL WITH OVERFITTING

- ▶ It's easier for more complex models to overfit
- ▶ Approaches to avoid overfitting
 - ▶ Regularization
 - ▶ Penalty term in scoring function
 - ▶ Model selection through cross-validation

NAIVE BAYES CLASSIFIERS

CLASSIFICATION AS PROBABILITY ESTIMATION

- ▶ Instead of learning a function f that assigns labels
- ▶ Learn a conditional probability distribution over the output of function f
- ▶ $P(f(x) \mid x) = P(f(x) = y \mid x_1, x_2, \dots, x_p)$
- ▶ Can use probabilities for the other two tasks
 - ▶ Classification
 - ▶ Ranking

KNOWLEDGE REPRESENTATION AND MODEL SPACE

BAYES RULE FOR PROBABILISTIC CLASSIFIER

$$P(C|\mathbf{X}) = \frac{P(\mathbf{X}|C)P(C)}{P(\mathbf{X})}$$

**BAYES
RULE**

$$= \frac{P(\mathbf{X}|C)P(C)}{[P(\mathbf{X}|C = +)P(C = +)] + [P(\mathbf{X}|C = -)P(C = -)]}$$

$$\propto P(\mathbf{X}|C)P(C)$$

**DENOMINATOR: NORMALIZING FACTOR
TO MAKE PROBABILITIES SUM TO 1
(CAN BE COMPUTED FROM NUMERATORS)**

NAIVE BAYES CLASSIFIER

$$P(C|\mathbf{X}) \propto P(\mathbf{X}|C)P(C)$$

**BAYES
RULE**

$$\propto \prod_{i=1}^m P(X_i|C)P(C)$$

**NAIVE
ASSUMPTION**

Assumption: Attributes are *conditionally independent* given the class

NBC LEARNING

$$\begin{aligned} P(BC|A, I, S, CR) &= \frac{P(A, I, S, CR|BC)P(BC)}{P(A, I, S, CR)} \\ &= \frac{P(A|BC)P(I|BC)P(S|BC)P(CR|BC)P(BC)}{P(A, I, S, CR)} \\ &\propto \frac{P(A|BC)P(I|BC)P(S|BC)P(CR|BC)P(BC)}{\end{aligned}$$

NBC parameters = CPDs+prior

- CPDs :
- P (A | BC)

P (I | BC)

P (S | BC)

P (CR | BC)

Prior:P (BC)

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

SCORING FUNCTION

LIKELIHOOD

- ▶ Let $D = \{x(1), \dots, x(n)\}$, where $x(i) = \langle \mathbf{x}_i, c_i \rangle$
- ▶ Assume the data D are independently sampled from the same distribution:

$$p(X|\theta)$$

- ▶ The likelihood function represents the probability of the data as a function of the model parameters:

$$\begin{aligned} L(\theta|D) &= L(\theta|x(1), \dots, x(n)) \\ &= p(x(1), \dots, x(n)|\theta) \\ &= \prod_{i=1}^n p(x(i)|\theta) \end{aligned}$$

**If instances are independent,
likelihood is product of probs**

LIKELIHOOD (CONT')

- ▶ Likelihood is not a probability distribution
 - ▶ Gives relative probability of data given a parameter
 - ▶ Numerical value of L is not relevant, only the ratio of two scores is relevant, e.g.,:

$$\frac{L(\theta_1|D)}{L(\theta_2|D)}$$

- ▶ **Likelihood function:** allows us to determine unknown parameters based on known outcomes
- ▶ **Probability distribution:** allows us to predict unknown outcomes based on known parameters

NBC: LIKELIHOOD

- ▶ NBC likelihood uses the NBC probabilities for each data instance (i.e., probability of the class given the attributes)

$$L(\theta|D) = \prod_{i=1}^n p(x(i) | \theta)$$

General likelihood

$$= \prod_{i=1}^n p(\mathbf{x}_i | c_i, \theta) P(c_i | \theta)$$

Product rule

$$= \prod_{i=1}^n \prod_{j=1}^m p(x_{ij} | c_i, \theta) P(c_i | \theta)$$

Naive assumption

SEARCH